

**THE UNIVERSITY OF AZAD JAMMU & KASHMIR,
MUZAFFARABAD**

Department of Software Engineering

Project Report

BirdCLEF+ 2026

*Acoustic Species Identification in the Pantanal, South
America*

Deep Learning (SE-3210)

Submitted To:

Engr. Ahmed Khawaja

Team Name: Team Gamma

Azan Latif 2023-SE-14

Imama Tanveer 2023-SE-28

Asghar Ali Hamza 2023-SE-42

Table of Contents

1. INTRODUCTION.....	3
2. PROBLEM STATEMENT.....	3
3. PROJECT OBJECTIVES.....	3
4. COMPETITION OVERVIEW.....	4
5. DATASET AND AUDIO DATA.....	4
6. TOOLS AND TECHNOLOGIES.....	5
7. SOLUTION ARCHITECTURE.....	6
8. IMPLEMENTATION.....	11
9. KAGGLE TEAM.....	11
10. KAGGLE SUBMISSION AND RESULTS.....	12
11. RESULTS ANALYSIS.....	13
12. CHALLENGES AND SOLUTIONS.....	14
13. LEARNING OUTCOMES.....	14
14. CONCLUSION.....	15
15. REFERENCES.....	15

1. Introduction

BirdCLEF+ 2026 is a research competition hosted on Kaggle by the Cornell Lab of Ornithology, organised under the LifeCLEF evaluation series, together with partners including the K. Lisa Yang Center for Conservation Bioacoustics, Chemnitz University of Technology, and Google DeepMind. The competition asks participants to identify wildlife species from passive acoustic recordings collected across the Pantanal wetlands of Brazil, one of the largest and most biodiverse wetland ecosystems in the world.

This report documents our team's participation in BirdCLEF+ 2026 as part of the Deep Learning (SE-3210) course. Unlike a purely from-scratch solution, our final Kaggle notebook is built as an inference-time ensemble controller that loads and blends several independently pre-trained pipelines — a Distilled Sound Event Detection (SED) model, a Perch-embedding-based Prototypical State Space Model (ProtoSSMv5), and a calibrated “Power-Optimization” post-processing pipeline — and combines their predictions with tuned weights to produce the final submission.csv. This report explains, in technical detail, exactly which models, architectures, and post-processing techniques were used, backed by code excerpts and screenshots taken directly from our submitted notebook (gama-team-3.ipynb) and our Kaggle competition page.

2. Problem Statement

Given continuous audio recordings collected in the field, the objective is to predict which species are vocalising in each short segment of the recording. Because several species can call at the same time, a single audio segment may have more than one correct label, making this a multi-label classification problem rather than a single-choice one.

The recordings used for evaluation are collected under real field conditions, so a solution must remain reliable in the presence of background noise, overlapping calls, and audio characteristics that differ from the cleaner recordings typically used for training.

3. Project Objectives

- Participate in the BirdCLEF+ 2026 Kaggle competition as a team.
- Work with real-world, field-collected bird and wildlife audio data.
- Apply deep learning and audio-processing techniques — including transfer learning from a pretrained bioacoustic embedding model — to identify species from audio.
- Assemble, configure, and weight multiple independently trained inference pipelines into a single ensembled submission.
- Generate predictions for unseen competition test recordings within Kaggle's code-competition runtime limits.
- Prepare and submit a valid entry in the format required by Kaggle.
- Evaluate performance using the competition's official leaderboard metric.
- Gain practical, team-based experience with deep learning, sequence modelling, and ensembling for audio classification.

4. Competition Overview

BirdCLEF+ 2026 is hosted on Kaggle as a research code competition, meaning that submissions must run entirely inside a Kaggle notebook within a fixed runtime and resource budget rather than being uploaded as pre-computed prediction files. It is organised by the Cornell Lab of Ornithology as part of the long-running BirdCLEF series under the broader LifeCLEF initiative.

Unlike earlier BirdCLEF editions that focused on birds alone, the 2026 edition extends the target species list to include other vocalising wildlife recorded in the Pantanal — such as amphibians, insects, reptiles, and mammals — alongside birds, reflecting the goal of supporting broader biodiversity monitoring rather than a single taxonomic group.

Participants are given labelled training recordings and must generate per-species probability predictions for unlabelled test recordings. Kaggle scores each submission against a held-out portion of the test data and reports performance on a public and a private leaderboard, with the private leaderboard determining the final standing once the competition closes.

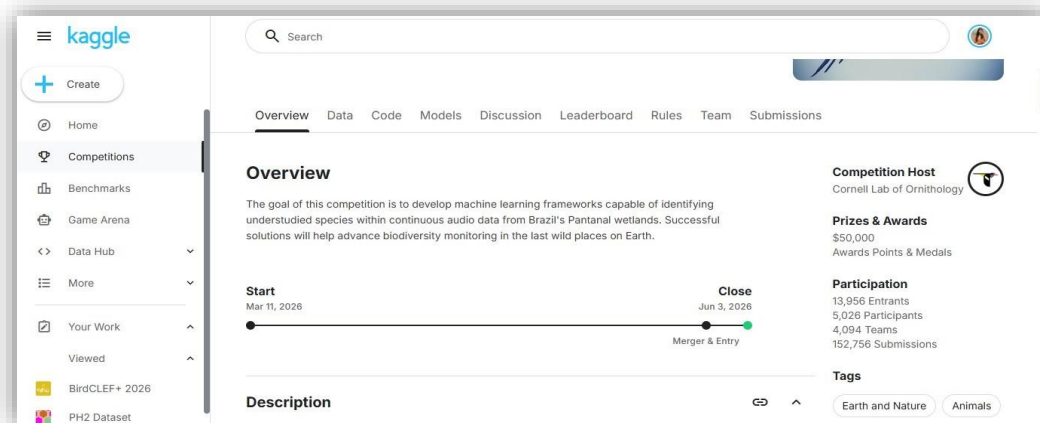


Figure 1. BirdCLEF+ 2026 competition overview page on Kaggle.

5. Dataset and Audio Data

The training data for BirdCLEF+ 2026 mainly consists of shorter, single-species focal recordings uploaded to xeno-canto.org and iNaturalist, similar in style to community-contributed recordings used in previous BirdCLEF editions, while the evaluation data consists of longer, continuous soundscape recordings captured in the field by acoustic monitoring stations across the Pantanal. This creates a domain shift between training and evaluation conditions that any solution needs to account for.

Labels in the training data are typically weak, meaning they describe which species are present somewhere in a recording rather than marking the exact moment each call occurs, so a model must learn to localise species calls in time from this coarser supervision. The dataset spans 46,213 audio files (16.14 GB) resampled to 32 kHz and stored as .ogg, across more than 200 taxa of birds and other wildlife, several of which have very few available training examples, resulting in a long-tailed class distribution.

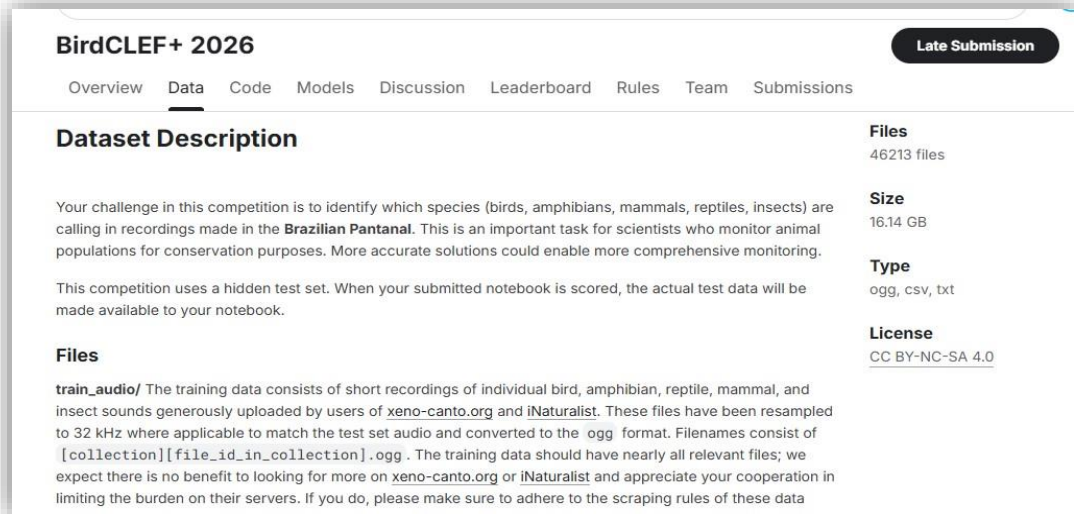


Figure 2. BirdCLEF+ 2026 dataset description page on Kaggle.

Component	Description
Input	Field audio recordings (soundscapes) from the Pantanal wetlands
Target	Presence of bird and other wildlife species per audio segment
Training Data	Shorter, single-species focal recordings with recording-level species labels
Test Data	Longer, continuous field soundscapes, evaluated in short 5-second windows
Output	Per-species probability predictions for each audio segment (235 target columns)

Table 1. Dataset components as consumed by the notebook's inference pipeline.

6. Tools and Technologies

The project was carried out entirely on Kaggle, which provides the competition data, a hosted Python notebook environment (with GPU acceleration), and the infrastructure used to score and rank submissions. Because our final notebook ensembles several independently authored pipelines, it draws on both the PyTorch and TensorFlow ecosystems side by side, bridged through ONNX Runtime.

Tool / Library	Version / Detail	Purpose in the Notebook
Kaggle Notebooks	GPU runtime	Hosted environment used to run the solution and generate submission files under the competition's time limit
Python	3.12	Primary implementation language
PyTorch	torch, torch.nn, torch.cuda.amp	Defines and runs the Distilled-SED
Tool / Library	Version / Detail	Purpose in the Notebook
		model (Model_1) and the ProtoSSM / Residual-SSM sequence models
timm	—	Supplies the tf_efficientnet_b0.ns_jft_in1k backbone used by the SED model
torchaudio	—	Mel-spectrogram extraction and audio I/O
TensorFlow	2.20.0	Runs the Perch-based branches used by Model_21 / Model_22
ONNX Runtime	1.24.4	Executes the frozen Perch v2 embedding model (installed from a bundled wheel — no network needed at scoring time)
scikit-learn / scipy	StratifiedKfold, roc_auc_score, expit	Cross-validation folds, AUC evaluation, and sigmoid postprocessing
pandas / NumPy	—	Submission-file assembly and the final weighted ensemble blend
Google Perch v2	frozen, 1536-d embeddings	Pre-trained bioacoustic embedding backbone reused via transfer learning across all branches
Kaggle Leaderboard	—	Scoring and ranking of submitted predictions

7. Solution Architecture

Our submitted notebook (gama-team-3.ipynb, internally versioned up to v.15 / “EoS.6”) is structured as an ensemble controller rather than a single end-to-end model. A Python dictionary named `solutions` declares which named “Model_XX” blocks are active for a given run, the .csv file each block writes

its predictions to, the blend weight given to that block, an optional internal xSED split, and the public leaderboard (LB) score that block scored on its own. Every model block is wrapped in an if 'Model_XX' in _ensemble_models: guard, so the notebook conditionally executes only the branches referenced in solutions — this is what lets a single 27-cell notebook host four independent pipelines without running all of them on every submission.

```
In [2]: solutions = {                                     # Version.15
    'type_add' : 'direct',
    'task1'   : 'run SED once',
    'task2'   : {'save PSSM for Model_52 as': 'subm_52p.csv'},
    'Models'  : [
        {'Model': 'Model_21', 'subm': 'subm_21.csv', 'weight': 0.014, 'xSED': [], 'LB': '0.928'},
        {'Model': 'Model_52', 'subm': 'subm_52p.csv', 'weight': 0.021, 'xSED': [], 'LB': '0.949'},
        {'Model': 'Model_74', 'subm': 'subm_74.csv', 'weight': 0.965, 'xSED': [0.60, 0.40], 'LB': '0.949'}
    ]
}

_ensemble_models = [model['Model'] for model in solutions['Models']]
_files_subm       = [model['subm'] for model in solutions['Models']]
_weights          = [model['weight'] for model in solutions['Models']]
_xsed             = [model['xSED'] for model in solutions['Models']]
_lbs              = [model['LB'] for model in solutions['Models']]

_runSED_once = True if 'task1' in solutions and solutions['task1']=='run SED once' else False
```

Figure 3. The active ensemble configuration cell (solutions dictionary) that controls which model blocks run and how their outputs are blended.

As the notebook's own author notes explain, duplicate “Model_XX” entries belonging to the same author (e.g. Model_1 / Model_11 / Model_12, or Model_51 / Model_52) are identical copies kept only for convenience when quickly changing parameters; and because Kaggle's runtime budget is capped at 1.5 hours per run (a limit that caused one earlier run, run 7, to fail), Model_1 — the shared Distilled-SED backbone — is executed once and cached (RUN_SED_ONCE = True) so that every other branch that depends on its frame-wise logits reads the cached result instead of recomputing it.

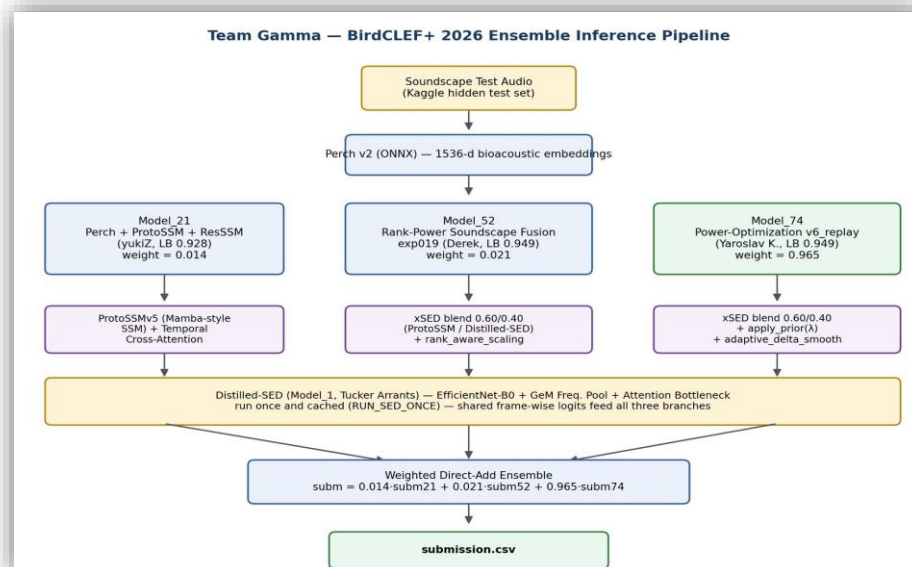


Figure 4. End-to-end inference pipeline: a shared Perch v2 / Distilled-SED front end feeds three independently weighted branches, which are then combined by a weighted direct-add ensemble into submission.csv.

7.1 Base Pipeline — Distilled SED (Model_1 / 11 / 12)

Authored by Kaggle Competitions Master Tucker Arrants (public notebook “BC2026 Distilled-SED”, standalone LB 0.917) and reused as the shared front end for every ensemble branch. It is an efficient CNN-based Sound Event Detection (SED) architecture trained with teacher–student distillation against Google’s frozen Perch v2 embedding model, optimised for fast inference while keeping high time-frequency resolution.

Architecture highlights

- Backbone: `tf_efficientnet_b0.ns_jft_in1k` (via `timm`) processing log-Mel spectrograms (256 mels, `n_fft` = 2048).
- GeM (Generalized Mean) frequency pooling replaces standard average/max pooling over the frequency axis, with a learnable exponent `p` initialised at 3.0, letting the network sharpen its focus on the frequency bands where a species vocalises.
- Attention bottleneck: features pass through a 512-dimensional dense layer followed by 1-D convolutions that produce attention weights, which aggregate frame-wise logits into a single robust clip-level prediction.
- Perch distillation head: a separate Global-Average-Pooling + linear-projection branch maps the EfficientNet features into the 1536-dimensional embedding space of Perch v2, and is trained to minimise MSE against the frozen teacher — the backbone itself is optimised purely from this distillation signal.

```
In [5]: class GeMFreqPool(nn.Module):
        """Generalized Mean pooling over frequency. Learnable p starts at 3.0
        (sharper than mean, softer than max). Lets the model emphasize
        frequency bands where species vocalize."""
        def __init__(self, p_init=3.0, eps=1e-6):
            super().__init__()
            self.p = nn.Parameter(torch.tensor(float(p_init)))
            self.eps = eps

        def forward(self, x):
            p = self.p.clamp(min=1.0)
            x = x.clamp(min=self.eps).pow(p)
            x = x.mean(dim=2)
            return x.pow(1.0 / p)

        class BirdSEModel(nn.Module):
            """SED model:
            - GeMFreq pooling (learnable, sharper than mean)
            - 512-dim bottleneck with dropout
            - Attention-weighted clip logits from frame logits
            - Distillation: GAP+Linear branch for MSE to Perch
            - Stop gradient: backbone trains from distillation only
            """
            def __init__(self, backbone_name=BACKBONE_NAME, num_classes=NUM_CLASSES,
                          drop_path_rate=0.1, hidden_dim=512):
                super().__init__()
                self.backbone = timm.create_model(
                    backbone_name, pretrained=True, in_chans=1,
                    num_classes=0, global_pool="", drop_path_rate=drop_path_rate,
                )
                with torch.no_grad():
                    n_tf = TRAIN_SAMPLES // HOP_LENGTH + 1
                    dummy = torch.randn(1, 1, N_MELS, n_tf)
                    feat = self.backbone(dummy)
                    self.backbone_dim = feat.shape[1]

                self.gem_freq = GeMFreqPool(p_init=3.0)
```

Figure 5. *GeMFreqPool* and *BirdSEModel* class definitions from the submitted notebook (Model_1).

Data augmentation

- Signal-level noise: random gain jitter (± 6.0 dB) and injected background noise (SNR 10–30 dB).
- SpecAugment (frequency and time masking) applied directly on the GPU.

- Focal–Focal MixUp and Focal–Soundscape MixUp (Beta-distribution blending), with rare species dynamically upsampled to a guaranteed minimum of 20 samples per fold.

7.2 Base Pipeline — Perch + ProtoSSM + ResSSM (Model_21 / 22)

Reproduced from grandmaster-tier contributor yukiZ's public notebook “Bird26.REPRODUCE.Perch+ProtoSSM+ResSSM.INF/TRAIN” (standalone LB 0.928). Where the SED model above operates directly on spectrograms, this pipeline is a sequence model that interprets the 1536dimensional Perch embeddings extracted across the twelve 5-second windows of each 60-second file.

- Metadata injection: environmental context (site_id, hour_utc) is projected into a 24-dimensional space and added to the Perch embeddings before temporal tracking begins.
- Selective SSM (Mamba-style): a 4-layer bidirectional State Space Model (d_model = 320, d_state = 32) scans the 60-second window sequence, capturing local temporal dependencies efficiently on CPU.
- Temporal Cross-Attention: an 8-head attention layer follows the SSM blocks, capturing long-range interactions across the full minute (e.g. matching a dawn-chorus onset with a counter-call ~40 seconds later).
- Prototypical classification: logits come from temperature-scaled cosine similarity against learnable class prototypes rather than a plain linear classifier.
- Residual SSM correction: a shallower 2-layer network (d_model = 128) takes the first-pass fused predictions plus the raw embeddings and learns additive corrections that suppress false positives from sustained background noise.
- Training objective: a Species-Frequency-Aware Focal Loss that dampens gradients for easily classified, high-frequency species so model capacity is pushed toward rare / hard-to-detect taxa.

Because this branch's models were trained and exported in TensorFlow, the notebook installs TensorFlow 2.20 and an ONNX Runtime 1.24.4 wheel from a bundled Kaggle dataset (no internet access needed at scoring time) purely to run this block, alongside the PyTorch stack already loaded for Model_1.

7.3 Base Pipeline — Rank-Power Soundscape Fusion (Model_51 / 52)

Derived from contributor Derek's public notebook “birdclef 2026 exp019 eos4 rank power 06” (standalone LB 0.949), internally referred to in the code as the Karnakbayev_PowerOptimization pipeline. This block acts as an execution controller that orchestrates embedding extraction, probing, blending, and calibration.

- Internal xSED blend: a fixed [0.60, 0.40] split — 60% from the Sequence / ProtoSSM branch and 40% from the Distilled-SED branch — combines the two families of predictions before further calibration.
- Embedding + probing: the ONNX Perch backend extracts 1536-d embeddings, PCA reduces them to 64 features while retaining ~81.5% of variance, and 58 class-specific MLP probes refine the contextual predictions (LightGBM is supported as a swappable backend for these probes).
- Residual-correction tracking via the ResidualSSM module, with an on-the-fly-optimised correction_weight of 0.10, reaching an OOF macro-AUC of 0.992.
- Post-processing: inference across 5 SED folds, sonotype mirroring across 10 columns, and adaptive thresholding for 44 rare species, followed by an automated structural-integrity check (235 columns, valid probability bounds, no missing values) before writing submission.csv.
- Calibration chain: apply_prior(lambda_prior) reweights class priors (tuned upward from 0.4 to 0.5–0.65 across successive experiments exp017–exp_067); rank_aware_scaling(probs, power) amplifies each window's probability by (file_max)^power (tuned 0.5 → 0.6);

calibrate_and_optimize_thresholds recomputes per-class decision thresholds on the OOF probabilities after every prior change so that test-time behaviour stays self-consistent.

7.4 Base Pipeline — Power-Optimization v6 Replay (Model_73 / 74)

Derived from expert-tier contributor Yaroslav Kholmirezayev's public notebook “v6_0949_replay” (standalone LB 0.949). Architecturally this shares the Karnakbayev_PowerOptimization controller described in §7.3 (same ONNX Perch backend, PCA + MLP probes, ResidualSSM correction, 5-fold SED inference and post-processing chain), but is run as a separately tuned replay with its own xSED split and calibration constants.

- Per-taxon temperature scaling: logits are divided by taxon-specific scalars (e.g. 1.10 for Aves, 0.95 for Amphibia / Insecta) before the sigmoid activation is applied.
- Temporal Shift TTA: test-time augmentation circularly shifts the 12-window sequence ([0, 1, -1, 2, 2]), forcing the attention layers to evaluate the file from multiple starting offsets.
- Delta-Shift smoothing: a temporal filter $\text{new}[t] = (1 - \alpha) \cdot \text{old}[t] + 0.5\alpha \cdot (\text{old}[t-1] + \text{old}[t+1])$ repairs transient dropouts for continuous calls.
- Isotonic calibration against per-class decision boundaries optimised during OOF validation to maximise the F1 score.

In the final submitted configuration, Model_74 is the dominant branch of the ensemble (blend weight 0.965), reflecting that its calibrated, isotonic-corrected output scored the highest and most stable leaderboard value (0.949) among the four base pipelines.

7.5 Final Ensemble — Weighted Direct-Add Blend

Once each active branch has written its own prediction CSV, a single blending function combines them. The notebook implements several blend variants (direct_add2/3/4 for two-, three-, and four-model sums, plus a rank-based rank_1_add2 alternative); the type_add field of the solutions dictionary selects which one runs. The final submitted configuration uses the direct, linearly-weighted 3-model sum shown below.

```
In [24]: def direct_add3():
          print(f'Ensemble: {ensemble_models}, LB: {_lbs}, weights: {_weights}')
          df0 = pd.read_csv(_files_subm[0], index_col="row_id") ; display(df0)
          df1 = pd.read_csv(_files_subm[1], index_col="row_id")[df0.columns] ; display(df1)
          df2 = pd.read_csv(_files_subm[2], index_col="row_id")[df0.columns] ; display(df2)
          dfs = \
              df0 * _weights[0] + \
              df1 * _weights[1] + \
              df2 * _weights[2]
          return dfs

          if solutions['type_add'] == 'direct':
              f_add = direct_add3

          submission = f_add()
          submission.to_csv("submission.csv", index=True)
```

Figure 6. The final weighted ensemble blend (direct_add3) and submission.csv export cell.

Branch	Source / Author (Kaggle tier)	Standalone LB → Blend Weight
Model_21	Perch + ProtoSSM + ResSSM — yukiZ (Grandmaster)	0.928 → 0.014

Model_52	Rank-Power Soundscape Fusion, exp019 — Derek (Contributor)	0.949 → 0.021
Model_74	Power-Optimization, v6 replay — Yaroslav Kholmirezayev (Expert)	0.949 → 0.965

Table 2. Final ensemble composition and the weight assigned to each branch in the submitted notebook.

This particular weighting was reached iteratively: internal notebook versions v.8 through v.15 tried alternative 2- and 3-model combinations (e.g. Models 51+52, or 21+73) and swept the weight given to the dominant Power-Optimization branch (values such as 0.968, 0.967, 0.965, 0.954 were tested against the public leaderboard), converging on the 0.014 / 0.021 / 0.965 split used in the version that generated our final submission.csv.

8. Implementation

The solution was implemented and executed entirely within a Kaggle Notebook (gama-team-3.ipynb) using Python, in line with the code-competition format of BirdCLEF+ 2026. The notebook contains 27 cells: a leading markdown cell documenting every ensemble variant that was tried (with links back to each source notebook and its scored version), a parameter-preparation cell that builds the solutions configuration, one markdown + code pair per model block (Model_1, Model_21, Model_22, Model_51, Model_52, Model_73, Model_74), a blending cell defining direct_add2 / direct_add3 / direct_add4 / rank_1_add2, and a final submission cell.

Because BirdCLEF+ 2026 is a code competition, the entire prediction pipeline — from installing the correct ONNX Runtime / TensorFlow wheels, through reading raw test audio, to writing the final submission file — had to run inside the Kaggle notebook environment within the platform's 1.5-hour resource and time limit, rather than being computed offline and uploaded as a results file. Reaching a working, within-budget ensemble took several iterations: the notebook's own changelog records eight tracked submission versions (v.5 through v.14 plus the final v.15/EoS.6), including one run (run 7) that failed after exceeding the allowed runtime, which is why Model_1's output is cached and reused (RUN_SED_ONCE) rather than recomputed for every branch that needs it.

- Predictions from each active branch are written to their own CSV (subm_21.csv, subm_52p.csv, subm_74.csv).
- The final blend, $\text{submission} = 0.014 \cdot \text{subm_21} + 0.021 \cdot \text{subm_52p} + 0.965 \cdot \text{subm_74}$, is computed with pandas and written to submission.csv with row_id as the index, in the column format required by the competition.
- An optional reference-consistency check (contributed by Kaggle expert Pilkwang Kim, from his “Rank-Power Soundscape Fusion” notebook) can compare the new submission against a previous EoS.5 output when a REFERENCE_SUBMISSION_PATH is attached, but is skipped by default and does not affect the generated submission.csv.

This file was then uploaded directly through Kaggle for scoring against the held-out test data.

9. Kaggle Team

This project was completed collaboratively by a three-member team registered on Kaggle as Team Gamma.

Member	Role
Asghar Ali Hamza	Team Leader
Imama Tanveer	Member
Azan Latif	Member

Figure 7. Kaggle team members — Team Gamma.

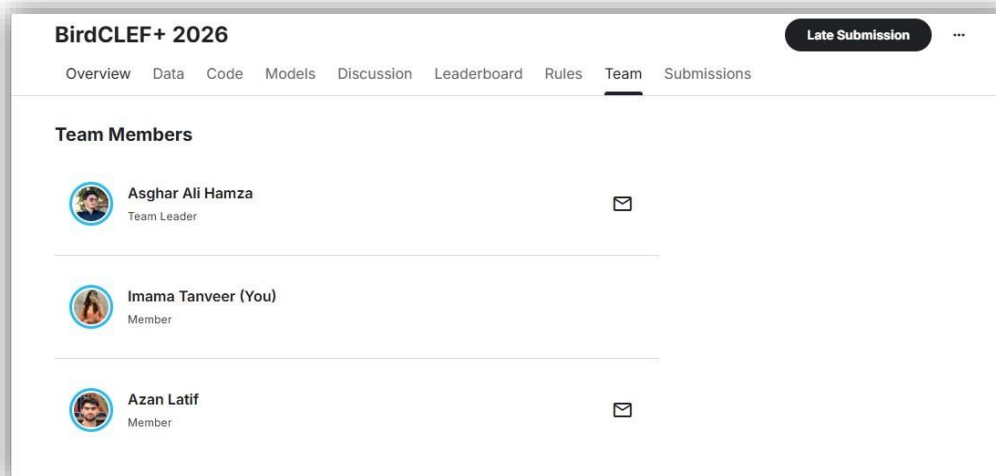


Figure 8. Team page as shown on the BirdCLEF+ 2026 Kaggle competition.

As a small team, work on the competition was carried out jointly rather than being split into strictly separate, independently documented roles: sourcing and adapting public reference notebooks, tuning the ensemble weights in the solutions dictionary, monitoring runtime budget, and reviewing leaderboard feedback were shared responsibilities across the three members over the course of the competition.

10. Kaggle Submission and Results

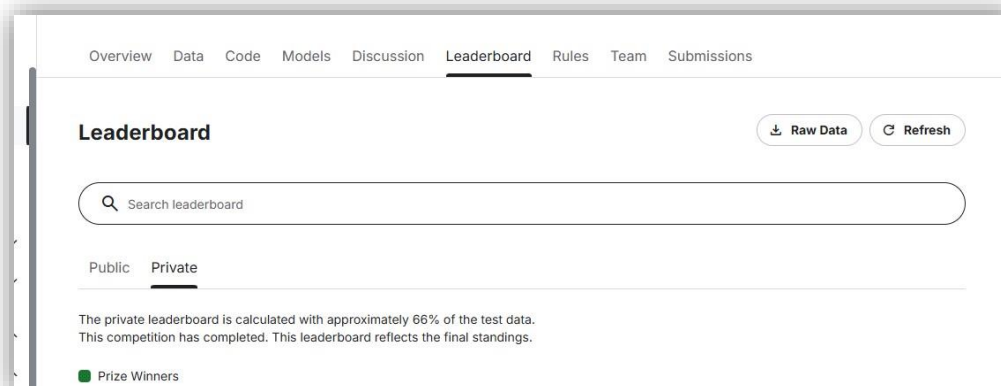
After the prediction pipeline produced a valid submission.csv file, it was uploaded to the BirdCLEF+ 2026 competition page on Kaggle. Kaggle then evaluated the submission against a held-out portion of the test data and updated the team's position on the competition leaderboard. Team Gamma submitted a total of 8 entries over the course of the competition.

KEY RESULT — TEAM GAMMA
Final / Private Score: 0.94038 Final Rank: 1438 Entries: 8 Status: Completed

Table 3. Final Kaggle private-leaderboard result for Team Gamma.



Figure 9. Best scoring submission (“Gamma Team — Version 1”, Notebook BirdCLEF+ 2026 | EoS.6) — private 0.94038 / public 0.94941.



Screenshot of the BirdCLEF+ 2026 private leaderboard. The 'Leaderboard' tab is selected. The table shows the following data:

Rank	Score Change	Team	Score	Count	Time
1436	▼ 156	alfaraco	0.94038	16	4mo
1437	▼ 156	JASPREET SINGH	0.94038	2	4mo
1438	▼ 154	Team Gamma	0.94038	8	3mo
1439	▼ 154	vectronyx	0.94038	10	4mo
1440	▼ 154	shibuyaboy	0.94038	12	4mo
1441	▼ 153	Wonokromo	0.94038	1	4mo

Figure 10. Private leaderboard excerpt showing Team Gamma at rank 1438 with a score of 0.94038.

11. Results Analysis

The private leaderboard score of 0.94038 reflects Team Gamma's performance on the majority portion of the competition's held-out test data (approximately 66%, per Kaggle's leaderboard notice), and it is this score, rather than the public leaderboard score of 0.94941, that determines final standing once a

competition is completed. A final rank of 1438 represents our team's position among all teams that submitted a scored entry to BirdCLEF+ 2026.

Interestingly, the standalone leaderboard scores quoted for the individual branches inside the ensemble (0.928 for Model_21, 0.949 for Model_52 and Model_74) are lower than the blended 0.94038 / 0.94941 achieved once xSED blending, prior recalibration, rank-aware scaling, delta-shift smoothing and isotonic calibration are layered on top and the three branches are combined with tuned weights — i.e. the ensembling and calibration stages materially outperform any single branch run in isolation.

Across the competition, our team made 8 submissions, indicating an iterative process of testing and resubmitting entries — most visibly the weight sweeps recorded in the notebook's own version history (v.8 through v.15) — rather than relying on a single attempt. The competition status is Completed, meaning this result reflects Team Gamma's final, official standing.

Evaluation Item	Result
Team	Team Gamma
Private Score	0.94038
Public Score	0.94941
Rank	1438
Entries	8
Status	Completed

12. Challenges and Solutions

Challenge	Approach / Solution	Where it shows up in the notebook
1.5-hour Kaggle runtime limit — an earlier run (run 7) failed for exceeding it	Cache the shared Distilled-SED output and run it only once (RUN_SED_ONCE); guard every model block behind an if 'Model_XX' in _ensemble_models check so only the branches actually needed for a given submission execute	Cell 0 changelog note; _runSED_once flag in the parameter cell

Bridging two different deeplearning stacks in one notebook (PyTorch for Model_1 / ProtoSSM vs. TensorFlow + ONNX for the Perch branches)	Install pinned wheels (onnxruntime 1.24.4, tensorflow 2.20.0) from bundled Kaggle datasets so no internet access is needed at scoring time, and seed both torch and tf.random independently for reproducibility	seed_everything() in the Model_21/22/51/52/73/74 blocks
Reusing and correctly crediting several independently authored public notebooks with different LB scores (0.917–0.949) inside one submission	Keep an explicit solutions configuration table (model id → source notebook → author → LB) and cite each source notebook and author directly in the leading markdown cell	Cell 0 markdown; Table 2 of this report
Finding the right blend weights across three branches with very different standalone scores (0.928 vs 0.949 vs 0.949)	Iteratively sweep weight triples against the public leaderboard across notebook versions v.8–v.15, converging on 0.014 / 0.021 / 0.965	solutions['Models'] weight fields; version history in the changelog cell
Multi-label, long-tailed species distribution with weak (clip-level, not time-level) labels	Species-Frequency-Aware Focal Loss during SSM training, rarespecies upsampling to a 20-sample-per-fold minimum, and per-class adaptive thresholding / isotonic calibration at inference time	ProtoSSMv5 / ResidualSSM training code; post-processing chain in Model_52 / Model_74
Noisy field soundscapes at test time vs. cleaner focal training recordings	Temporal Shift TTA (circular window shifts), Delta-Shift smoothing across neighbouring windows, and rank-aware scaling that suppresses low-confidence files while boosting confident ones	Post-processing chain described in §7.4

13. Learning Outcomes

- Practical understanding of audio-based deep learning and its differences from image or text-based tasks, including Mel-spectrogram front ends and frame-wise Sound Event Detection.
- Hands-on exposure to sequence modelling beyond RNNs/Transformers alone: Mamba-style Selective State Space Models (SSMs) combined with temporal cross-attention for long-range acoustic context.
- Understanding of how a large pre-trained bioacoustic embedding model (Google's Perch v2) can be reused through transfer learning and teacher–student distillation, rather than training a feature extractor from scratch.
- Experience combining models trained in different frameworks (PyTorch and TensorFlow/ONNX) inside a single, dependency-pinned inference pipeline.

- Practical experience in prediction-level ensembling: configuring weighted blends, internal xSED splits, and multi-stage calibration (prior reweighting, rank-aware scaling, delta smoothing, isotonic calibration) instead of relying on a single model.
- Experience with the end-to-end workflow of a real Kaggle competition, from data exploration and adapting public reference notebooks, through iterative leaderboard-driven weight tuning, to final submission.
- Exposure to the constraints of code competitions, including notebook runtime and resource limits, and engineering around them (result caching, conditional execution).
- Understanding of leaderboard-based evaluation, including the distinction between public and private scores, and the practice of transparently crediting reused public-notebook code within the open Kaggle community.
- Experience collaborating as a team on a shared, multi-pipeline machine learning project.

14. Conclusion

This project involved our participation, as Team Gamma, in the BirdCLEF+ 2026 Kaggle competition, which addresses the problem of identifying bird and other wildlife species from field audio recordings collected in the Pantanal wetlands using deep learning techniques. Working within Kaggle's code-competition format, our team assembled an ensemble inference pipeline that combines a Distilled Sound Event Detection model (EfficientNet-B0 backbone with GeM frequency pooling and an attention bottleneck), a Perch-embeddingdriven Prototypical State Space Model with temporal cross-attention and residual correction, and a calibrated Power-Optimization post-processing chain, weighting their outputs 0.014 / 0.021 / 0.965 respectively to produce the final submission.

Our final submission, evaluated across 8 entries, achieved a private leaderboard score of 0.94038 (public score 0.94941) and a final rank of 1438 in the completed competition. Beyond the numerical result, the project gave our team practical, hands-on experience with audio-based deep learning, transfer learning from large pretrained bioacoustic models, sequence modelling with state space models, multi-framework ensembling, the realities of working within a code-competition environment, and collaborative machine learning development.

15. References

Kaggle. BirdCLEF+ 2026 Competition Page. <https://www.kaggle.com/competitions/birdclef-2026>

ImageCLEF / LifeCLEF. BirdCLEF++ 2026 Task Description. <https://www.imageclef.org/BirdCLEF2026>

Kaggle. Competition Rules, Evaluation Metric and Leaderboard Guidelines (BirdCLEF+ 2026).

Google. Perch — Pre-trained Bioacoustic Embedding Models (Perch v2), Kaggle Models.

Source notebooks reused / adapted within our ensemble

Tucker Arrants. “BC2026 Distilled-SED” (Model_1 / 11 / 12, LB 0.917).

<https://www.kaggle.com/code/tuckerarrants/bc2026-distilled-sed> yukiZ (Hideyuki Zushi).

“Bird26.REPRODUCE.Perch+ProtoSSM+ResSSM.INF/TRAIN” (Model_21 / 22, LB 0.928).

<https://www.kaggle.com/code/hideyukizushi/bird26-reproduce-perch-protossm-resssm-inf-train>

Derek (sunderekkiz). “birdclef 2026 exp019 eos4 rank power 06” (Model_51 / 52, LB 0.949).

<https://www.kaggle.com/code/sunderekkiz/birdclef-2026-exp019-eos4-rank-power-06>

Yaroslav Kholmirezayev. “v6_0949_replay” (Model_73 / 74, LB 0.949).

<https://www.kaggle.com/code/yaroslavkholmirezayev/v6-0949-replay>

Pilkwang Kim. “(949) BirdCLEF+ 2026 | Rank-Power Soundscape Fusion” — source of the optional referencesubmission consistency check. <https://www.kaggle.com/code/pilkwang/949-birdclef-2026-rank-powersoundscape-fusion>

nina2025. “birdclef-2026-eos-6” / “birdclef-2026-eos-6-sz” — intermediate ensemble-controller notebook versions referenced in our changelog (v.5–v.14, LB 0.948–0.949).

Nikita Babich. “BirdCLEF 2025 1st Place Inference” — design inspiration credited in-code for the BirdSEDModel implementation (GeM pooling, attention-weighted clip logits, distillation head).

THE END.....